
Sleep & Wakeup

Synchronization through sleep&wakeup

Contents

- Busy waiting vs. Block & Wakeup
- Producer/Consumer Problem
- Implementation `sleep()` & `wakeup()` in xv6
- Applications of `sleep()` & `wakeup()` in xv6
 - `sleeplock`
 - `pause`
 - `pipe`

Busy waiting vs. Block & Wakeup

Software Platforms Lab.

Semaphore Implementation

- Must guarantee that no two processes can execute `wait ()` and `signal ()` on the same semaphore at the same time
 - Thus, implementation becomes the critical section problem where the wait and signal code are placed in the critical section
 - We can use the Peterson's algorithm, for example, to synchronize concurrent accesses to P() and V()
- Using **busy waiting** in P():
 - Implementation code is short
 - Little busy waiting if critical section rarely occupied
 - Note that applications may spend lots of time in critical sections and therefore this is not a good solution
 - May spend long time in P()

19

Software Platforms Lab.

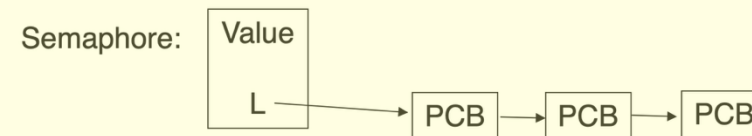
Block / Wakeup Implementation

- **Permits multiple processes to enter the wait function**
- Define a semaphore as a record

```
typedef struct {  
    int          value;    /* semaphore */  
    struct process *L;      /*process wait queue*/  
} semaphore;
```

- Assume two simple operations:

- **block** kernel suspends the process that invoked itself. Put this process' PCB into wait queue (semaphore)
- **wakeup(P)** resumes the execution of a blocked process P. (Put this process' PCB into ready queue)



20

How xv6 Provides Block & Wakeup in Kernel Code

- `void sleep(void *chan, struct spinlock *lk)`
 - Puts the current process to sleep on the channel.
- `void wakeup(void *chan)`
 - Wakes up all processes sleeping on the channel.

Producer / Consumer Problem

From Busy waiting to Block & Wakeup

Producer / Consumer with Busy Waiting

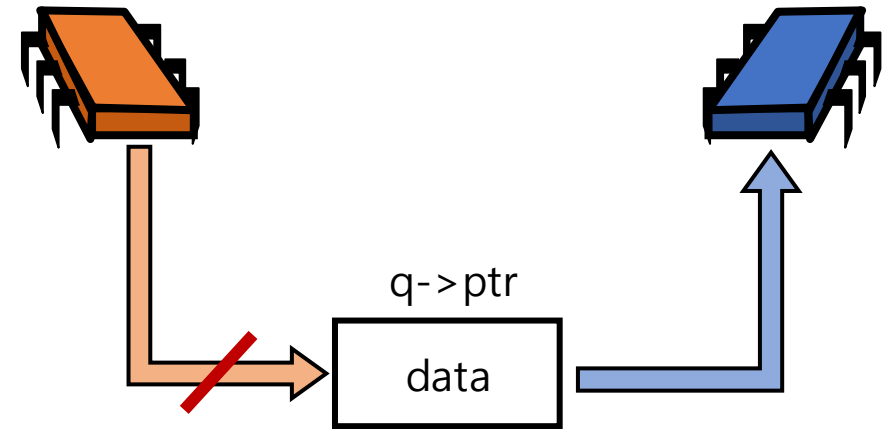
```
100 struct q {  
101     void *ptr;  
102 };  
103  
104 void* send(struct q *q, void *p) {  
105     while(q->ptr != 0);  
106     q->ptr = p;  
107 }  
108  
109 void* recv(struct q *q) {  
110     void *p;  
111     while((p = q->ptr) == 0);  
112     q->ptr = 0;  
113     return p;  
114 }
```

Producer writes a data into q
if q is empty

Consumer reads a data from
q
if q is not empty.
After read a data, clear the q

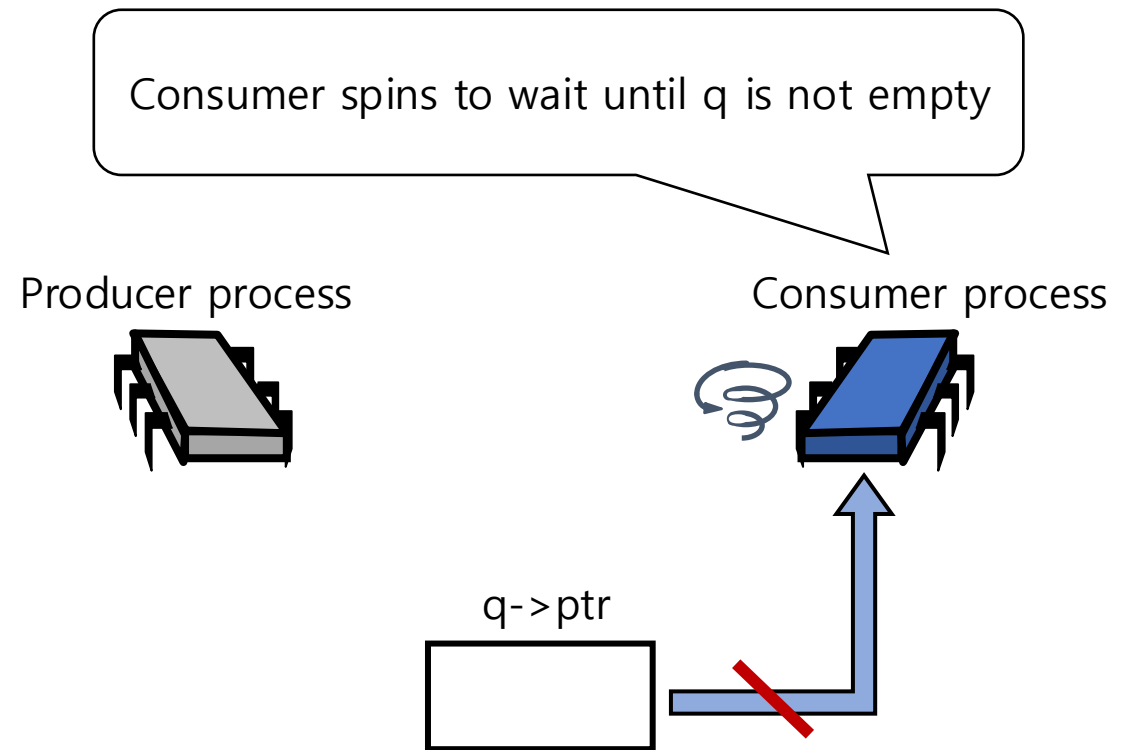
Producer process

Consumer process



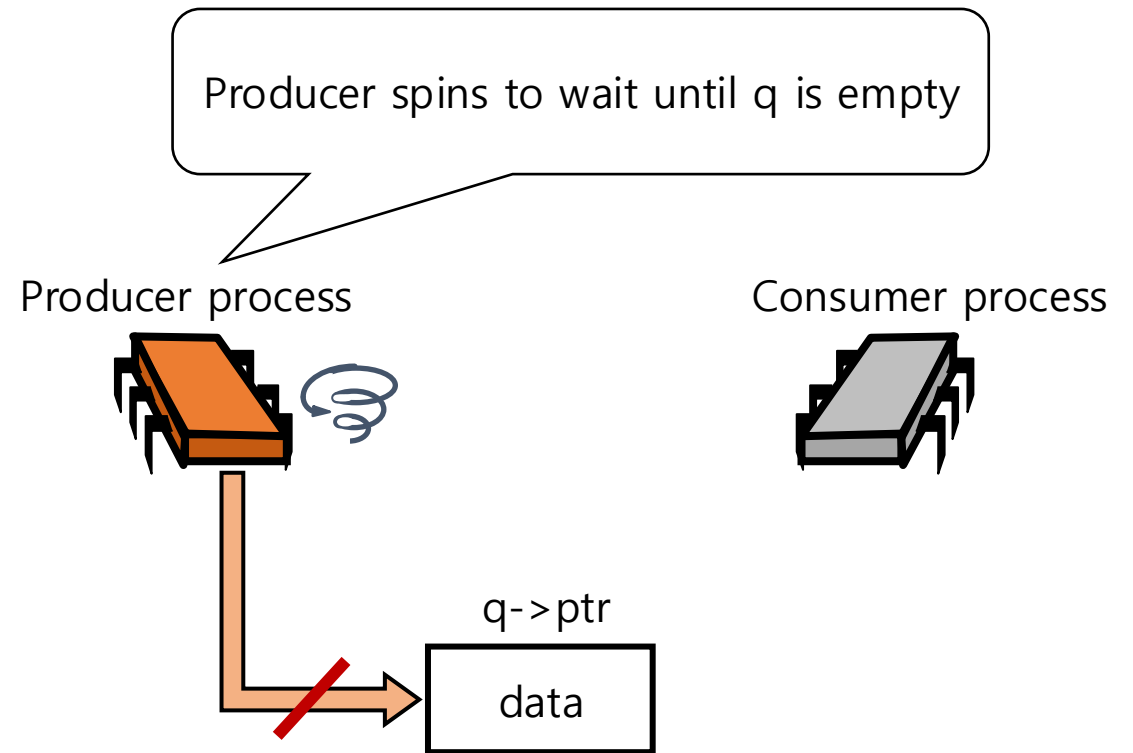
Producer / Consumer with Busy Waiting

```
100 struct q {  
101     void *ptr;  
102 };  
103  
104 void* send(struct q *q, void *p) {  
105     while(q->ptr != 0);  
106     q->ptr = p;  
107 }  
108  
109 void* recv(struct q *q) {  
110     void *p;  
111     while((p = q->ptr) == 0);  
112     q->ptr = 0;  
113     return p;  
114 }
```



Producer / Consumer with Busy Waiting

```
100 struct q {  
101     void *ptr;  
102 };  
103  
104 void* send(struct q *q, void *p) {  
105     while(q->ptr != 0);  
106     q->ptr = p;  
107 }  
108  
109 void* recv(struct q *q) {  
110     void *p;  
111     while((p = q->ptr) == 0);  
112     q->ptr = 0;  
113     return p;  
114 }
```



Producer / Consumer with Busy Waiting

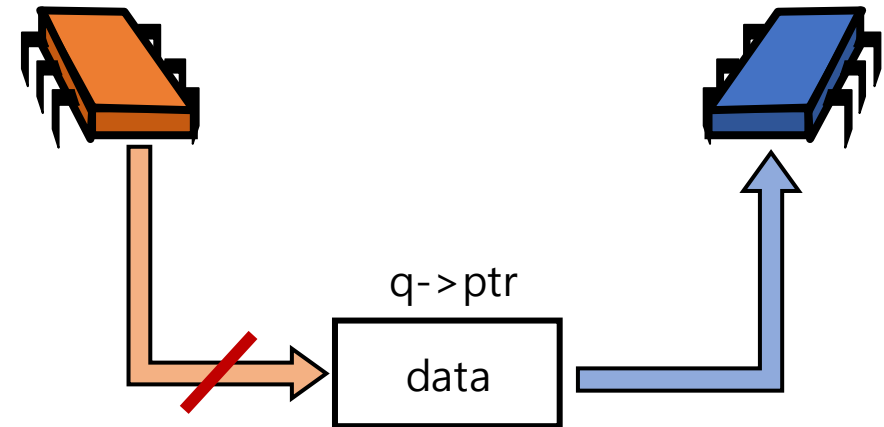
```
100 struct q {  
101     void *ptr;  
102 };  
103  
104 void* send(struct q *q, void *p) {  
105     while(q->ptr != 0);  
106     q->ptr = p;  
107 }  
108  
109 void* recv(struct q *q) {  
110     void *p;  
111     while((p = q->ptr) == 0);  
112     q->ptr = 0;  
113     return p;  
114 }
```

It works, but too expensive

Spinning process does nothing,
but wastes CPU time

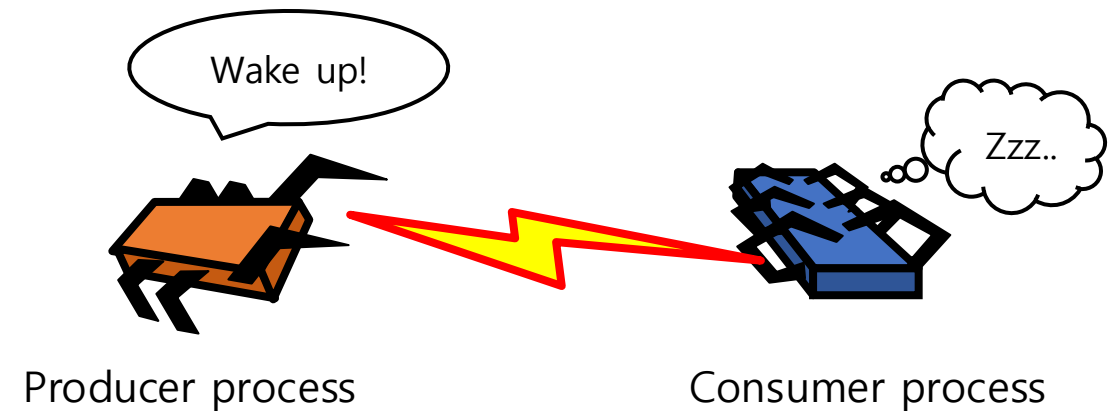
Producer process

Consumer process



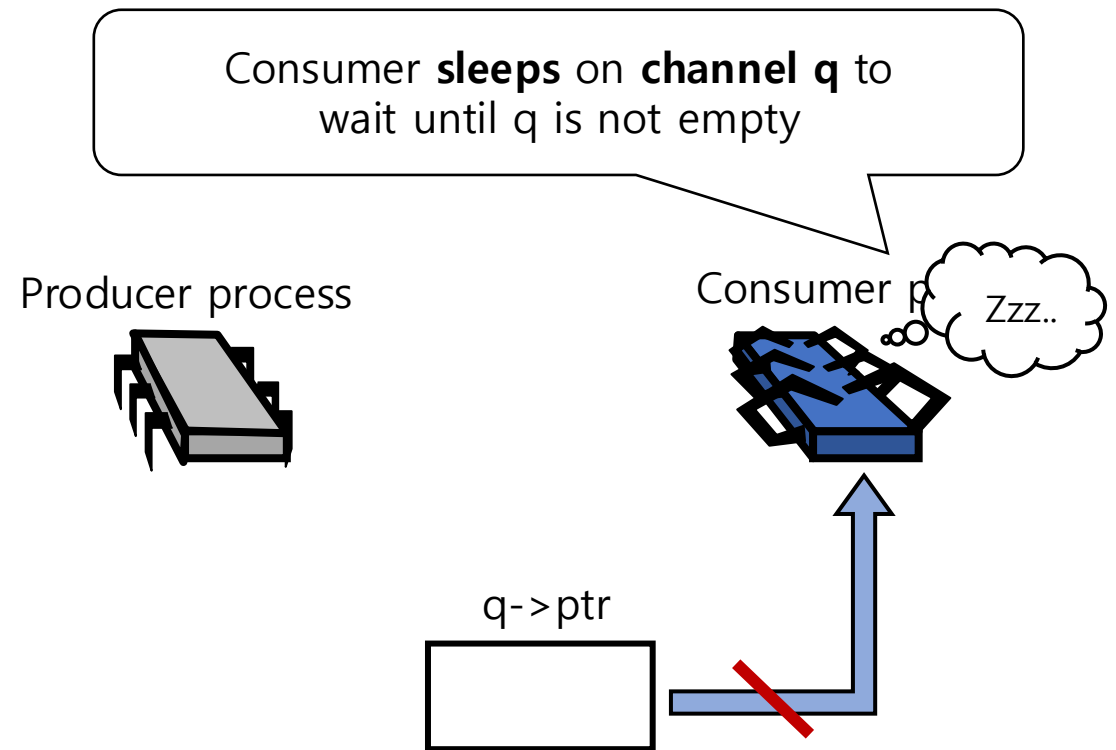
Producer / Consumer with Sleep & Wakeup

```
200 struct q {
201     void *ptr;
202 };
203
204 void* send(struct q *q, void *p) {
205     while(q->ptr != 0);
206     q->ptr = p;
207     wakeup(q); /* wake recv */
208 }
209
210 void* recv(struct q *q) {
211     void *p;
212     while((p = q->ptr) == 0)
213         sleep(q);
214     q->ptr = 0;
215     return p;
216 }
```



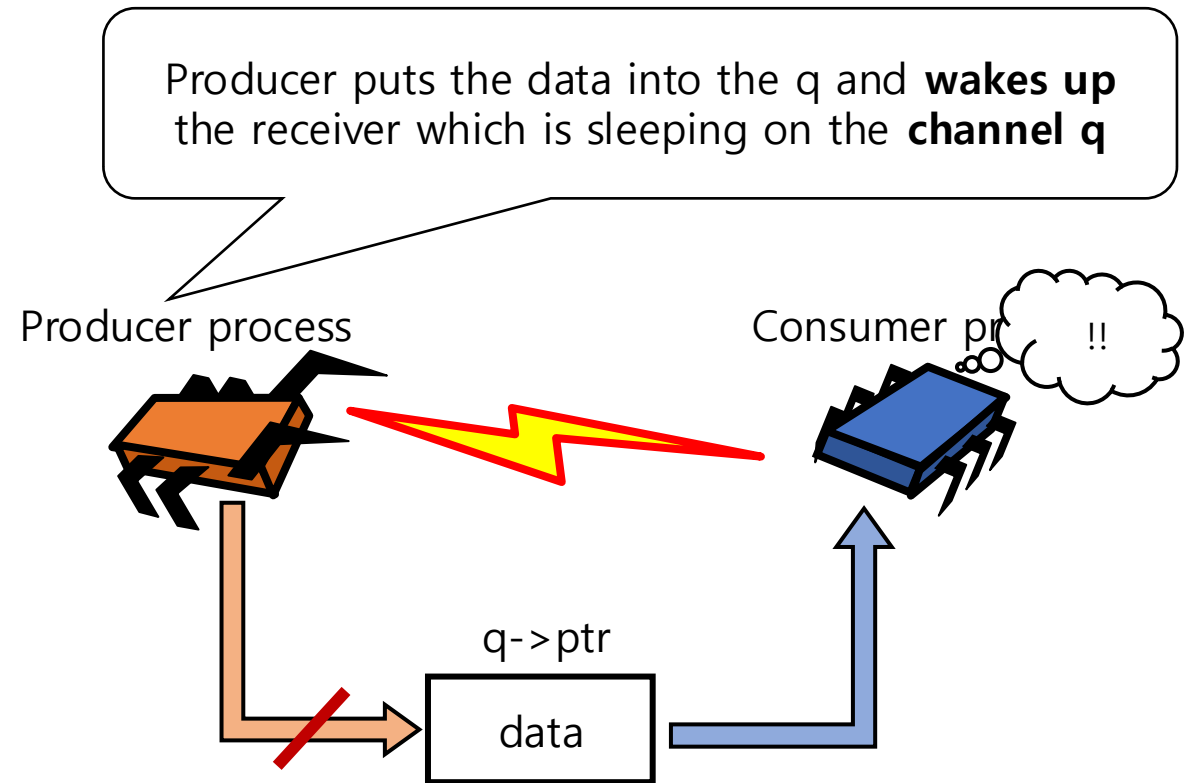
Producer / Consumer with Sleep & Wakeup

```
200 struct q {
201     void *ptr;
202 };
203
204 void* send(struct q *q, void *p) {
205     while(q->ptr != 0);
206     q->ptr = p;
207     wakeup(q); /* wake recv */
208 }
209
210 void* recv(struct q *q) {
211     void *p;
212     while((p = q->ptr) == 0)
213         sleep(q);
214     q->ptr = 0;
215     return p;
216 }
```



Producer / Consumer with Sleep & Wakeup

```
200 struct q {
201     void *ptr;
202 };
203
204 void* send(struct q *q, void *p) {
205     while(q->ptr != 0);
206     q->ptr = p;
207     wakeup(q); /* wake recv */
208 }
209
210 void* recv(struct q *q) {
211     void *p;
212     while((p = q->ptr) == 0)
213         sleep(q);
214     q->ptr = 0;
215     return p;
216 }
```



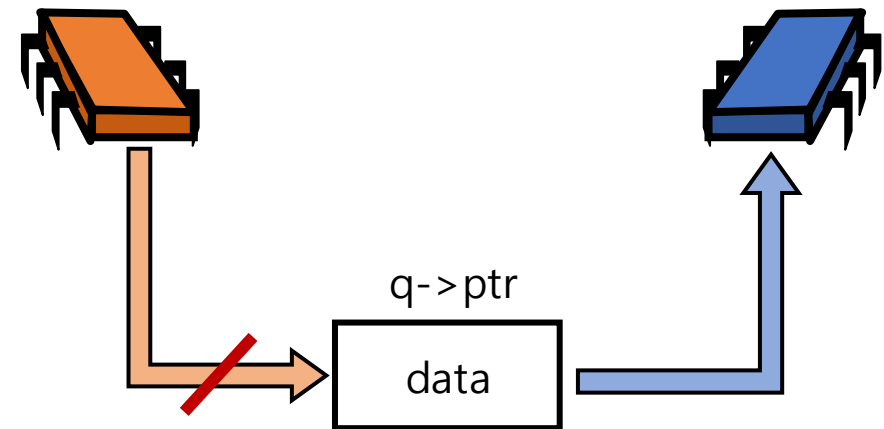
Producer / Consumer with Sleep & Wakeup

```
200 struct q {
201     void *ptr;
202 };
203
204 void* send(struct q *q, void *p) {
205     while(q->ptr != 0);
206     q->ptr = p;
207     wakeup(q); /* wake recv */
208 }
209
210 void* recv(struct q *q) {
211     void *p;
212     while((p = q->ptr) == 0)
213         sleep(q);
214     q->ptr = 0;
215     return p;
216 }
```

Consumer doesn't need to waste CPU time by sleeping,
but..

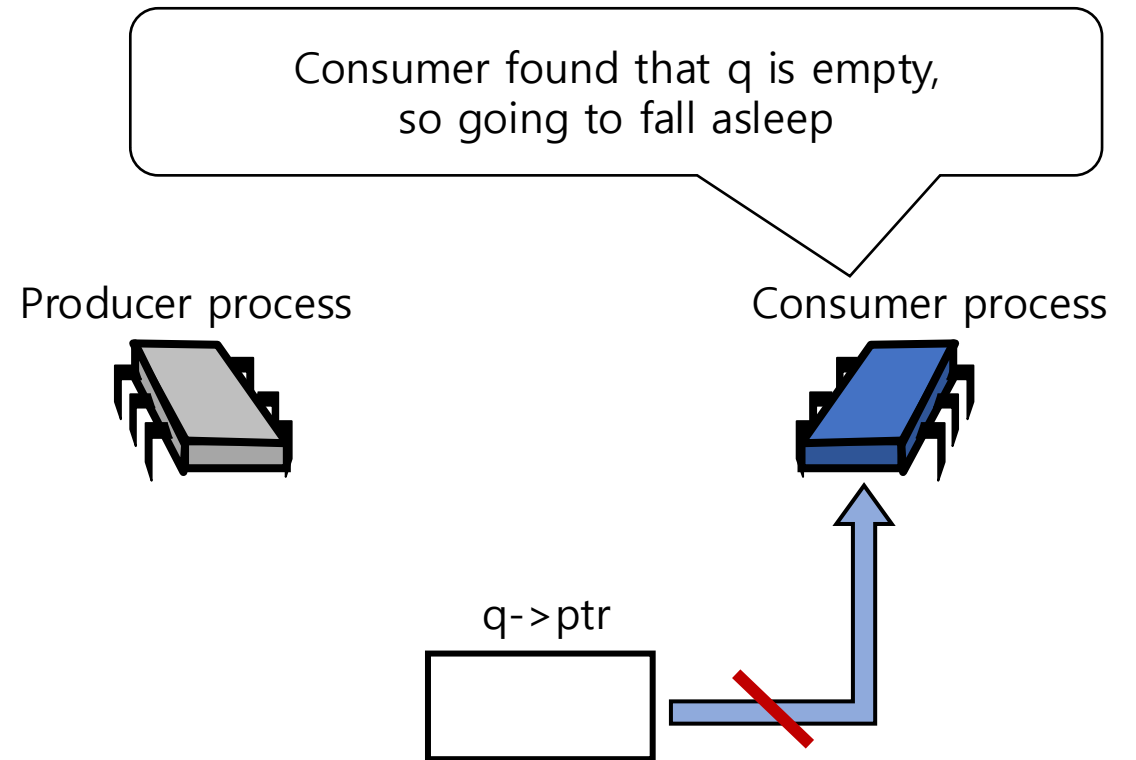
Producer process

Consumer process



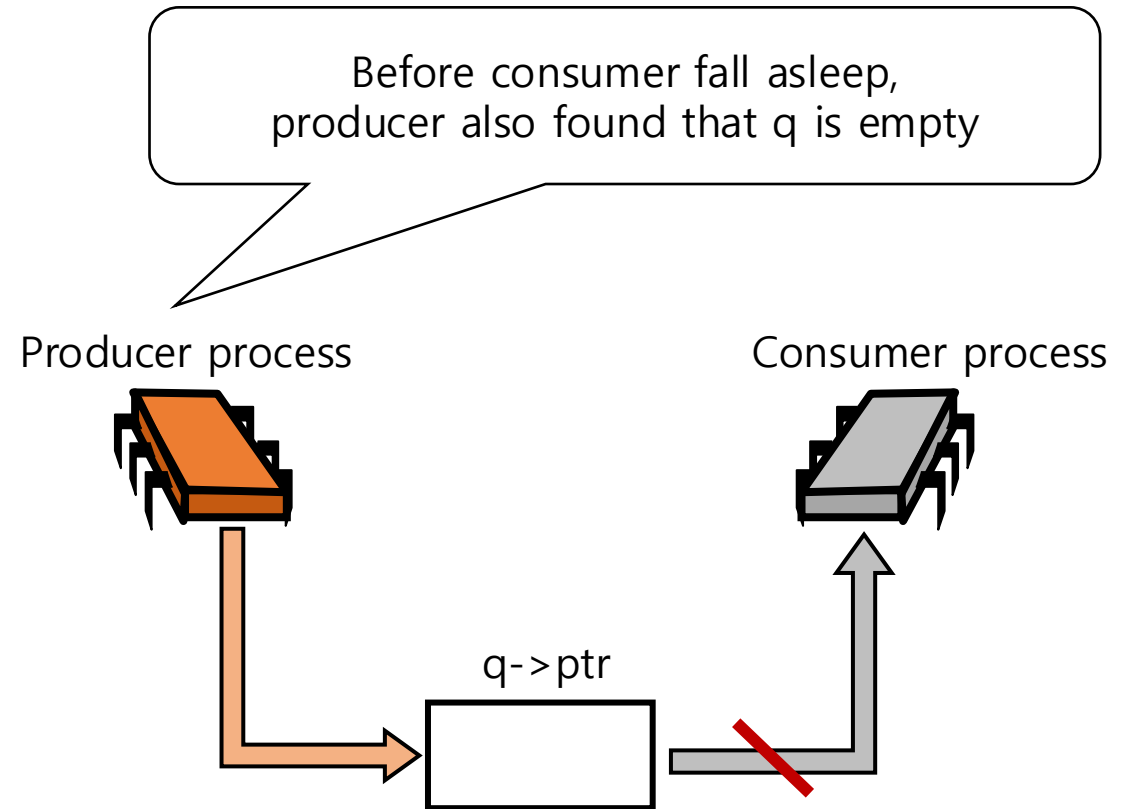
Lost wake-up problem

```
200 struct q {
201     void *ptr;
202 };
203
204 void* send(struct q *q, void *p) {
205     while(q->ptr != 0);
206     q->ptr = p;
207     wakeup(q); /* wake recv */
208 }
209
210 void* recv(struct q *q) {
211     void *p;
212     while((p = q->ptr) == 0)
213         sleep(q);
214     q->ptr = 0;
215     return p;
216 }
```



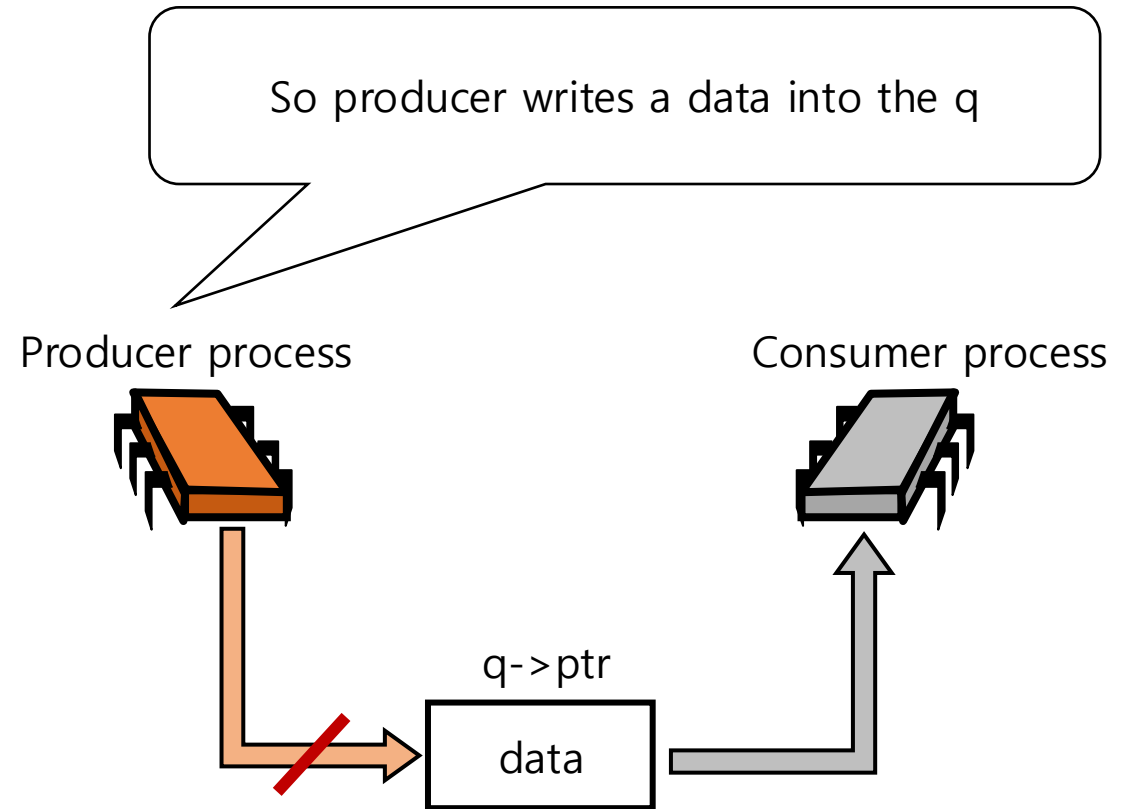
Lost wake-up problem

```
200 struct q {
201     void *ptr;
202 };
203
204 void* send(struct q *q, void *p) {
205     while(q->ptr != 0);
206     q->ptr = p;
207     wakeup(q); /* wake recv */
208 }
209
210 void* recv(struct q *q) {
211     void *p;
212     while((p = q->ptr) == 0)
213         sleep(q);
214     q->ptr = 0;
215     return p;
216 }
```



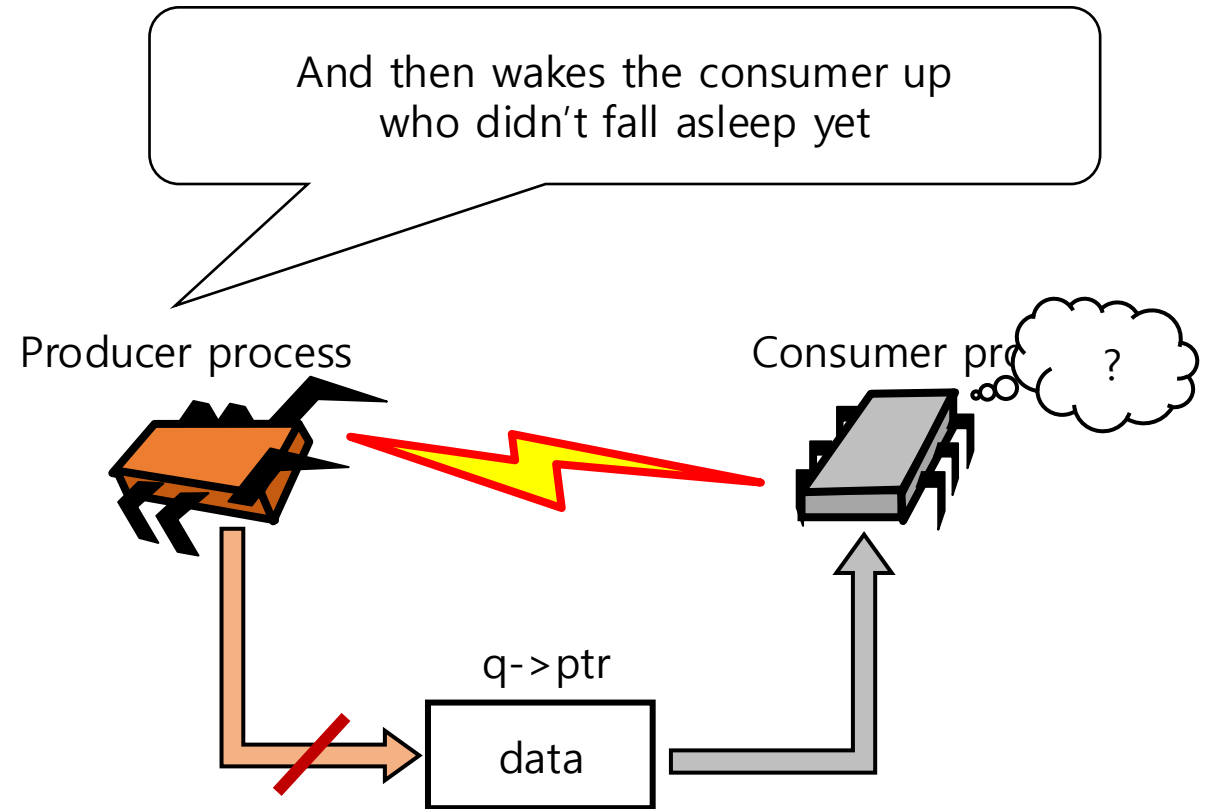
Lost wake-up problem

```
200 struct q {
201     void *ptr;
202 };
203
204 void* send(struct q *q, void *p) {
205     while(q->ptr != 0);
206     q->ptr = p;
207     wakeup(q); /* wake recv */
208 }
209
210 void* recv(struct q *q) {
211     void *p;
212     while((p = q->ptr) == 0)
213         sleep(q);
214     q->ptr = 0;
215     return p;
216 }
```



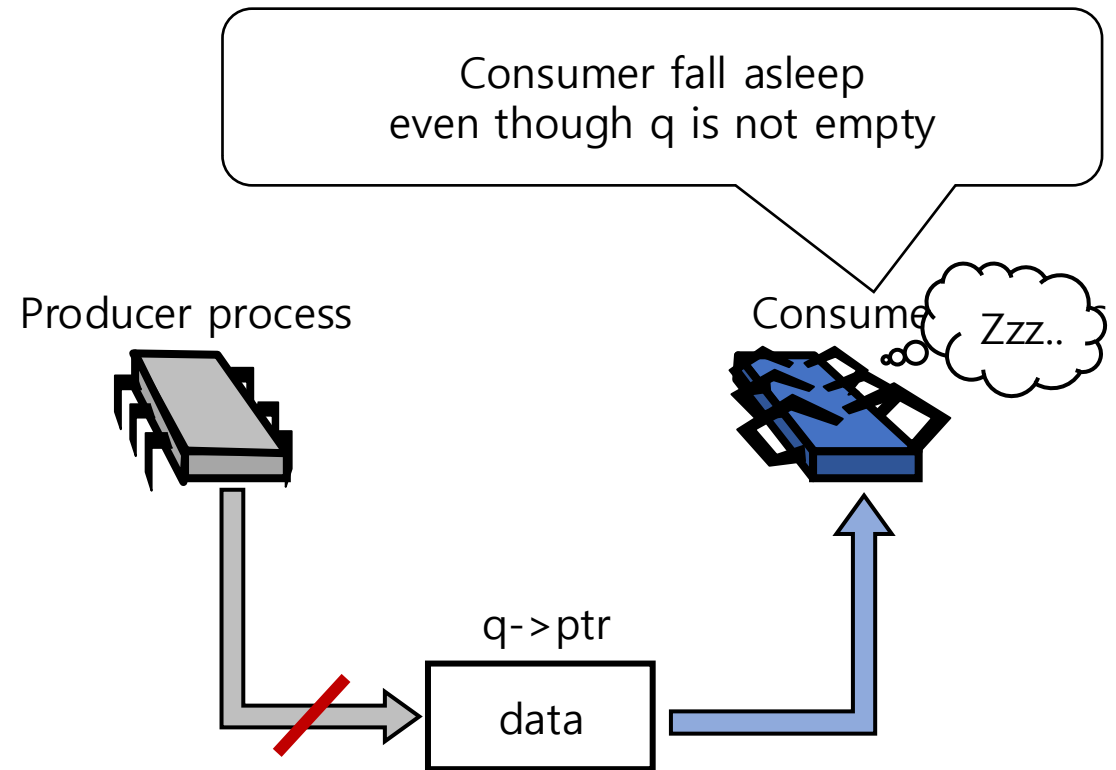
Lost wake-up problem

```
200 struct q {
201     void *ptr;
202 };
203
204 void* send(struct q *q, void *p) {
205     while(q->ptr != 0);
206     q->ptr = p;
207     wakeup(q); /* wake recv */
208 }
209
210 void* recv(struct q *q) {
211     void *p;
212     while((p = q->ptr) == 0)
213         sleep(q);
214     q->ptr = 0;
215     return p;
216 }
```



Lost wake-up problem

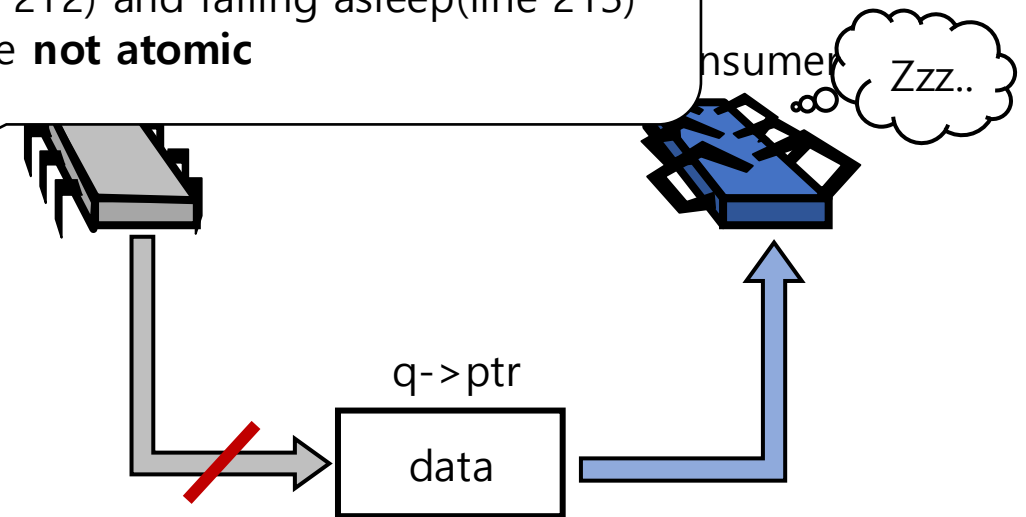
```
200 struct q {
201     void *ptr;
202 };
203
204 void* send(struct q *q, void *p) {
205     while(q->ptr != 0);
206     q->ptr = p;
207     wakeup(q); /* wake recv */
208 }
209
210 void* recv(struct q *q) {
211     void *p;
212     while((p = q->ptr) == 0)
213         sleep(q);
214     q->ptr = 0;
215     return p;
216 }
```



Lost wake-up problem

```
200 struct q {
201     void *ptr;
202 };
203
204 void* send(struct q *q,
205     void *p;
206     while(q->ptr != 0);
207     q->ptr = p;
208     wakeup(q); /* wake rec
209 }
210
211 void* recv(struct q *q) {
212     void *p;
213     while((p = q->ptr) == 0)
214         sleep(q);
215     q->ptr = 0;
216     return p;
217 }
```

Lost wake-up occurs because
checking q's state(line 212) and falling asleep(line 213)
are **not atomic**



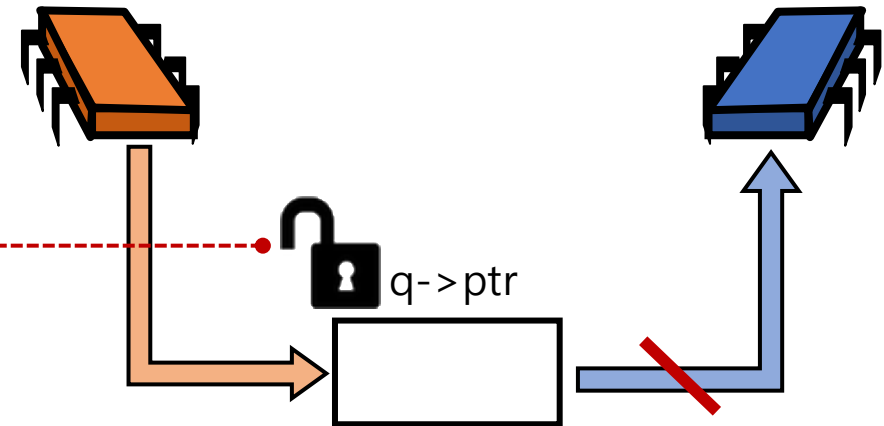
Sleep & Wakeup with Spin Lock

```
300 struct q {  
301     struct spinlock lock;  
302     void *ptr;  
303 };  
304  
305 void* send(struct q *q, void *p) {  
306     acquire(&q->lock);  
307     while(q->ptr != 0);  
308     q->ptr = p;  
309     wakeup(q);  
310     release(&q->lock);  
311 }  
312  
313 void* recv(struct q *q) {  
314     void *p;  
315     acquire(&q->lock);  
316     while((p = q->ptr) == 0)  
317         sleep(q);  
318     q->ptr = 0;  
319     release(&q->lock);  
320     return p;  
321 }
```

So, we can use a spinlock to run those operations atomically

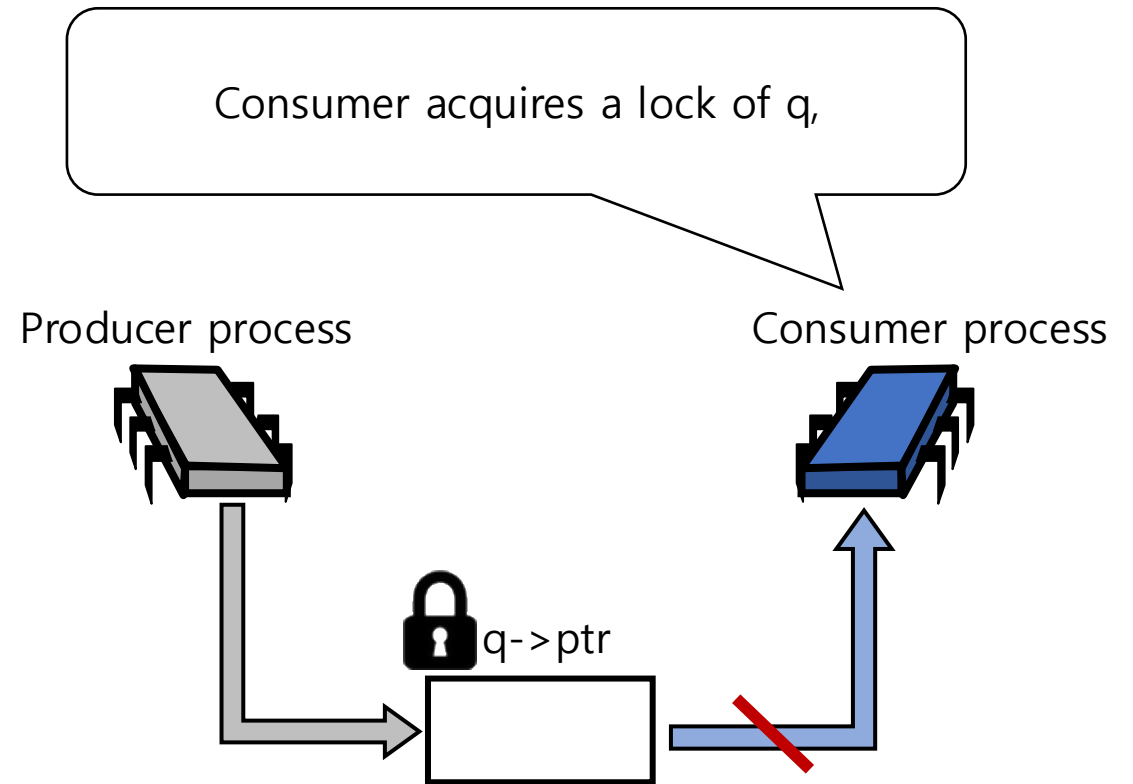
Producer process

Consumer process



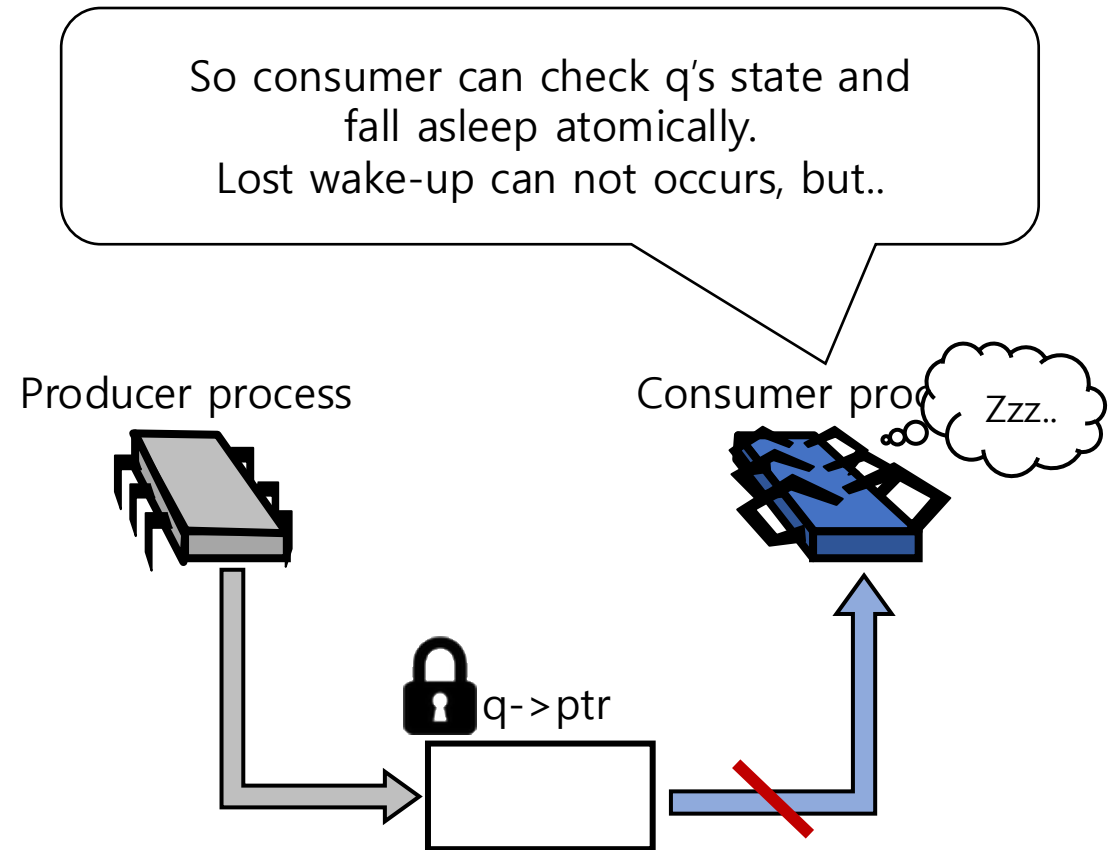
Sleep & Wakeup with Spin Lock

```
300 struct q {
301     struct spinlock lock;
302     void *ptr;
303 };
304
305 void* send(struct q *q, void *p) {
306     acquire(&q->lock);
307     while(q->ptr != 0);
308     q->ptr = p;
309     wakeup(q);
310     release(&q->lock);
311 }
312
313 void* recv(struct q *q) {
314     void *p;
315     acquire(&q->lock);
316     while((p = q->ptr) == 0)
317         sleep(q);
318     q->ptr = 0;
319     release(&q->lock);
320     return p;
321 }
```



Sleep & Wakeup with Spin Lock

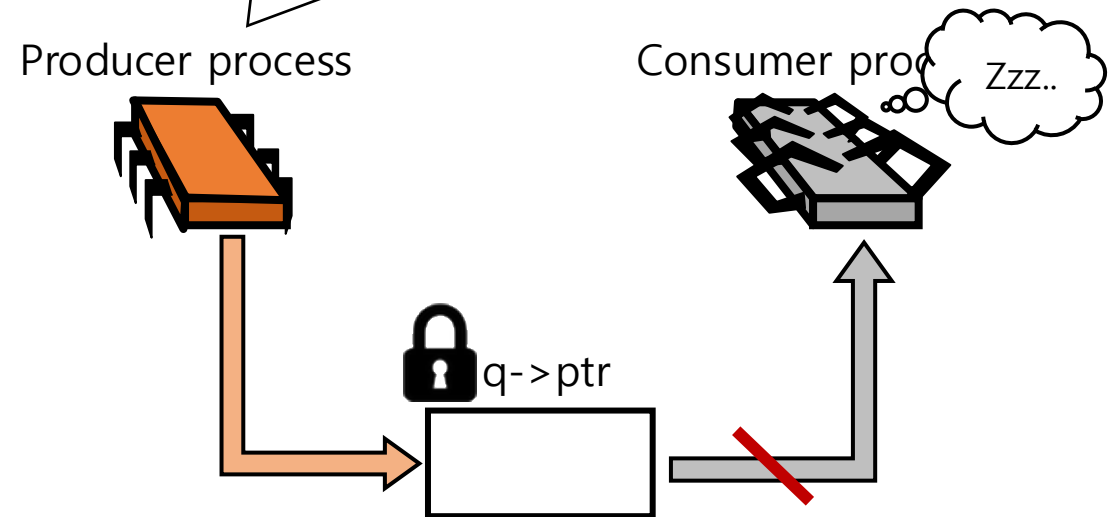
```
300 struct q {
301     struct spinlock lock;
302     void *ptr;
303 };
304
305 void* send(struct q *q, void *p) {
306     acquire(&q->lock);
307     while(q->ptr != 0);
308     q->ptr = p;
309     wakeup(q);
310     release(&q->lock);
311 }
312
313 void* recv(struct q *q) {
314     void *p;
315     acquire(&q->lock);
316     while((p = q->ptr) == 0)
317         sleep(q);
318     q->ptr = 0;
319     release(&q->lock);
320     return p;
321 }
```



Sleep & Wakeup with Spin Lock

```
300 struct q {
301     struct spinlock lock;
302     void *ptr;
303 };
304
305 void* send(struct q *q, void *p) {
306     acquire(&q->lock);
307     while(q->ptr != 0);
308     q->ptr = p;
309     wakeup(q);
310     release(&q->lock);
311 }
312
313 void* recv(struct q *q) {
314     void *p;
315     acquire(&q->lock);
316     while((p = q->ptr) == 0)
317         sleep(q);
318     q->ptr = 0;
319     release(&q->lock);
320     return p;
321 }
```

Producer waits for consumer to release q's lock.
Consumer is waiting for sender to wake it up,
so **dead-lock have been occured**

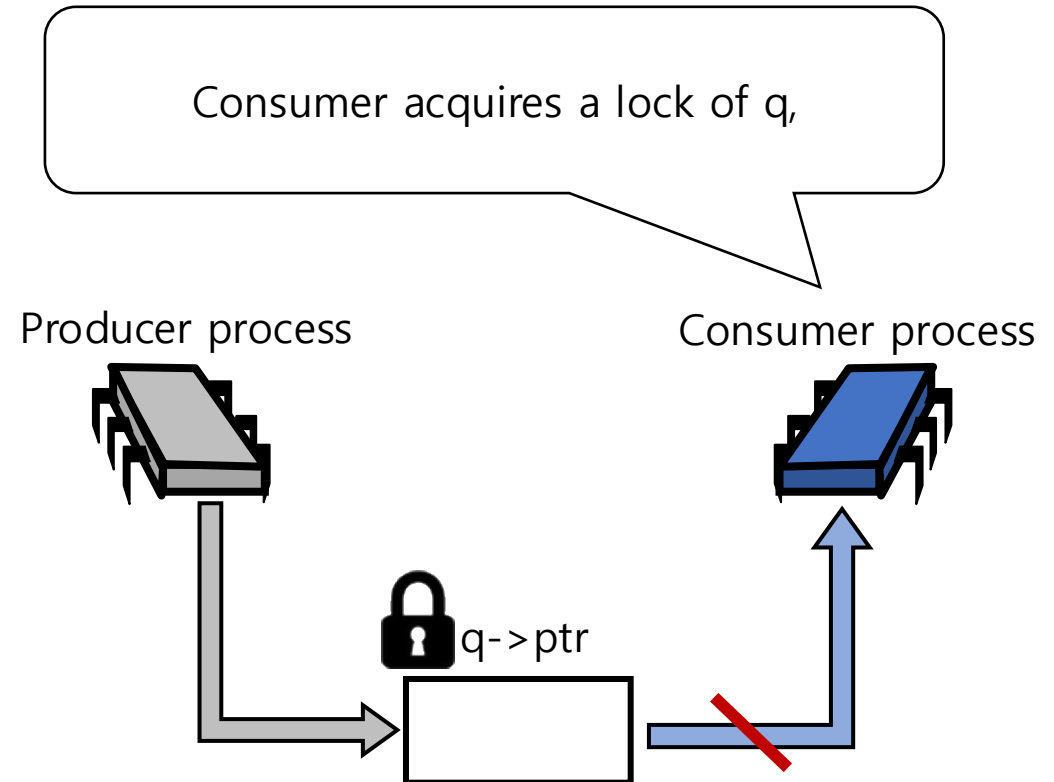


How xv6 Provides Block & Wakeup in Kernel Code

- `void sleep(void *chan, struct spinlock *lk)`
 - Puts the current process to sleep on the channel.
 - **Atomically releases lock before sleeping and reacquires it when awakened.**
- `void wakeup(void *chan)`
 - Wakes up all processes sleeping on the channel.

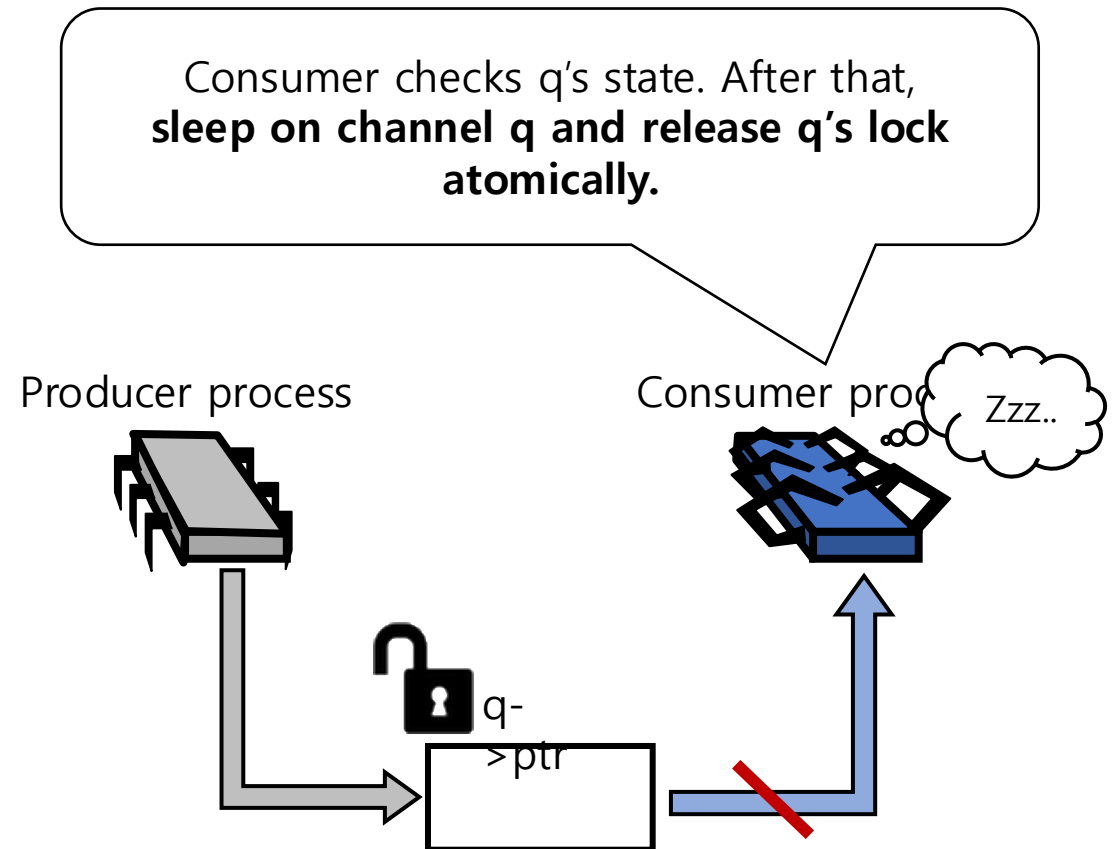
Sleep & Wakeup in xv6

```
300 struct q {
301     struct spinlock lock;
302     void *ptr;
303 };
304
305 void* send(struct q *q, void *p) {
306     acquire(&q->lock);
307     while(q->ptr != 0);
308     q->ptr = p;
309     wakeup(q);
310     release(&q->lock);
311 }
312
313 void* recv(struct q *q) {
314     void *p;
315     acquire(&q->lock);
316     while((p = q->ptr) == 0)
317         sleep(q, &q->lock);
318     q->ptr = 0;
319     release(&q->lock);
320     return p;
321 }
```



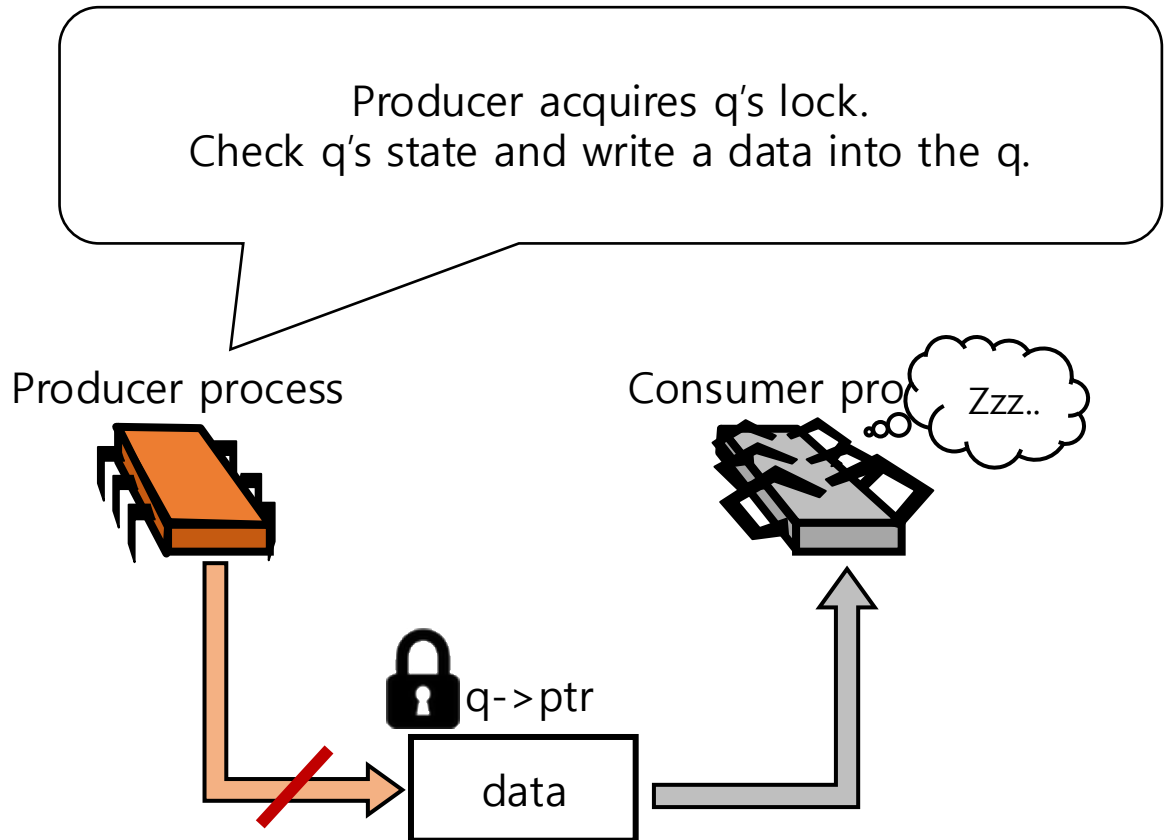
Sleep & Wakeup in xv6

```
300 struct q {
301     struct spinlock lock;
302     void *ptr;
303 };
304
305 void* send(struct q *q, void *p) {
306     acquire(&q->lock);
307     while(q->ptr != 0);
308     q->ptr = p;
309     wakeup(q);
310     release(&q->lock);
311 }
312
313 void* recv(struct q *q) {
314     void *p;
315     acquire(&q->lock);
316     while((p = q->ptr) == 0)
317         sleep(q, &q->lock);
318     q->ptr = 0;
319     release(&q->lock);
320     return p;
321 }
```



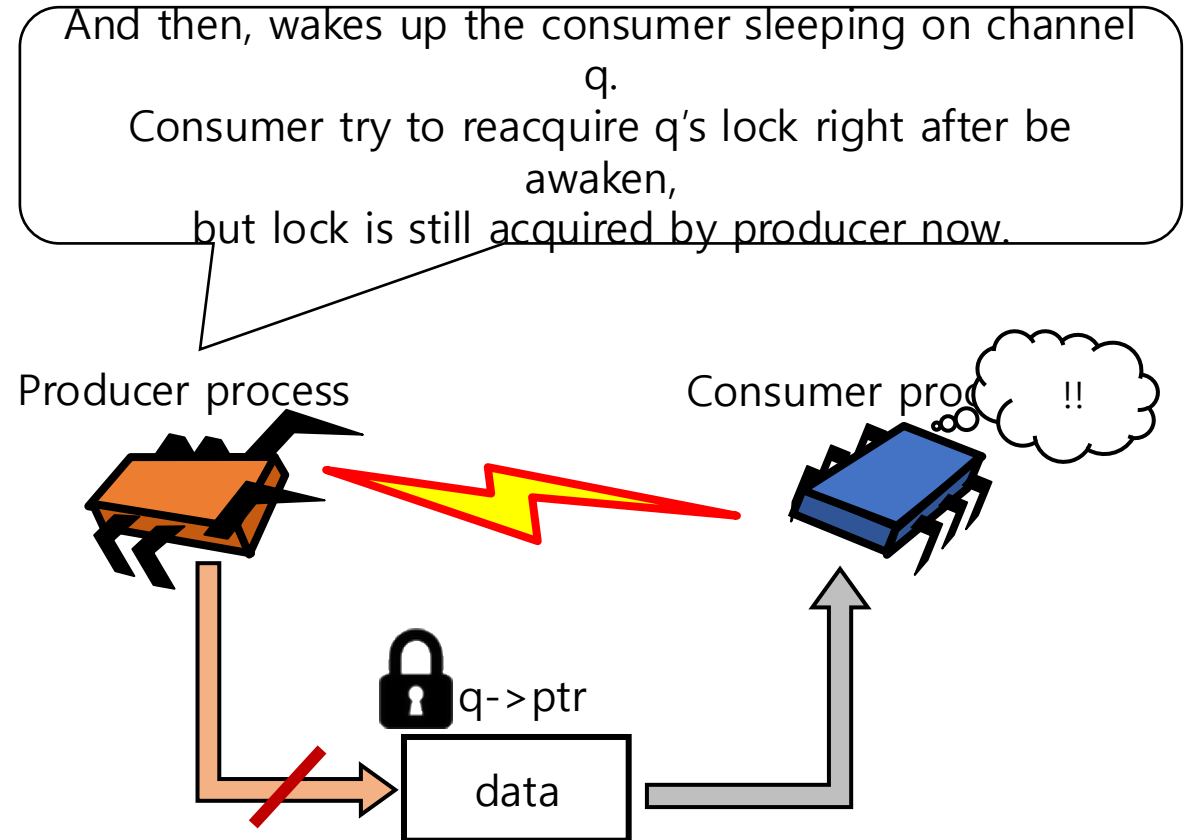
Sleep & Wakeup in xv6

```
300 struct q {
301     struct spinlock lock;
302     void *ptr;
303 };
304
305 void* send(struct q *q, void *p) {
306     acquire(&q->lock);
307     while(q->ptr != 0);
308     q->ptr = p;
309     wakeup(q);
310     release(&q->lock);
311 }
312
313 void* recv(struct q *q) {
314     void *p;
315     acquire(&q->lock);
316     while((p = q->ptr) == 0)
317         sleep(q, &q->lock);
318     q->ptr = 0;
319     release(&q->lock);
320     return p;
321 }
```



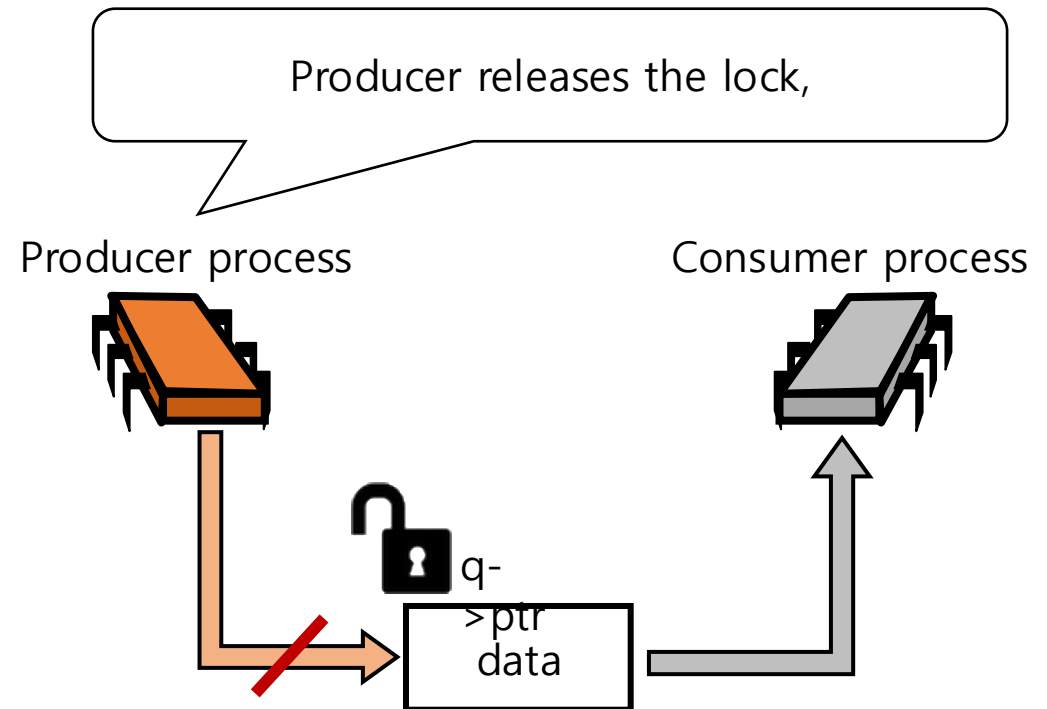
Sleep & Wakeup in xv6

```
300 struct q {
301     struct spinlock lock;
302     void *ptr;
303 };
304
305 void* send(struct q *q, void *p) {
306     acquire(&q->lock);
307     while(q->ptr != 0);
308     q->ptr = p;
309     wakeup(q);
310     release(&q->lock);
311 }
312
313 void* recv(struct q *q) {
314     void *p;
315     acquire(&q->lock);
316     while((p = q->ptr) == 0)
317         sleep(q, &q->lock);
318     q->ptr = 0;
319     release(&q->lock);
320     return p;
321 }
```



Sleep & Wakeup in xv6

```
300 struct q {
301     struct spinlock lock;
302     void *ptr;
303 };
304
305 void* send(struct q *q, void *p) {
306     acquire(&q->lock);
307     while(q->ptr != 0);
308     q->ptr = p;
309     wakeup(q);
310     release(&q->lock);
311 }
312
313 void* recv(struct q *q) {
314     void *p;
315     acquire(&q->lock);
316     while((p = q->ptr) == 0)
317         sleep(q, &q->lock);
318     q->ptr = 0;
319     release(&q->lock);
320     return p;
321 }
```



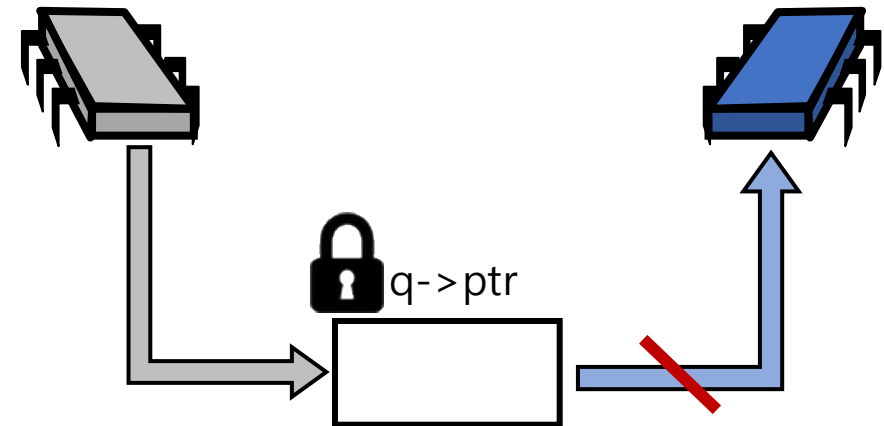
Sleep & Wakeup in xv6

```
300 struct q {
301     struct spinlock lock;
302     void *ptr;
303 };
304
305 void* send(struct q *q, void *p) {
306     acquire(&q->lock);
307     while(q->ptr != 0);
308     q->ptr = p;
309     wakeup(q);
310     release(&q->lock);
311 }
312
313 void* recv(struct q *q) {
314     void *p;
315     acquire(&q->lock);
316     while((p = q->ptr) == 0)
317         sleep(q, &q->lock);
318     q->ptr = 0;
319     release(&q->lock);
320     return p;
321 }
```

So consumer now able to reacquire the q's lock.
After that, read a data from q and clear the q

Producer process

Consumer process



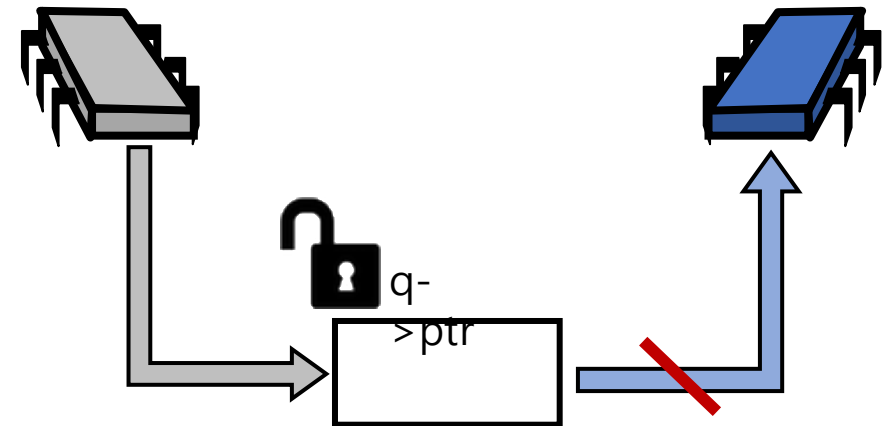
Sleep & Wakeup in xv6

```
300 struct q {
301     struct spinlock lock;
302     void *ptr;
303 };
304
305 void* send(struct q *q, void *p) {
306     acquire(&q->lock);
307     while(q->ptr != 0);
308     q->ptr = p;
309     wakeup(q);
310     release(&q->lock);
311 }
312
313 void* recv(struct q *q) {
314     void *p;
315     acquire(&q->lock);
316     while((p = q->ptr) == 0)
317         sleep(q, &q->lock);
318     q->ptr = 0;
319     release(&q->lock);
320     return p;
321 }
```

Now the producer/consumer model works well!

Producer process

Consumer process



sleep() & wakeup()

Implementation in xv6

sleep()

```
$ vim kernel/proc.c
```

```
542 void
543 sleep(void *chan, struct spinlock *lk)
544 {
545     struct proc *p = myproc();
546     ...
547     acquire(&p->lock); //DOC: sleeplock1
548     release(lk);
549
550     // Go to sleep.
551     p->chan = chan;
552     p->state = SLEEPING;
553
554     sched();
555
556     ...
557 }
```

- sleep()
 - Release a lock and sleep on the channel.
 - Reacquires lock when awakened.

sleep()

```
$ vim kernel/proc.c
```

```
542 void
543 sleep(void *chan, struct spinlock *lk)
544 {
545     struct proc *p = myproc();
546     ...
547     sched();
548     // Tidy up.
549     p->chan = 0;
550     // Reacquire original lock.
551     release(&p->lock);
552     acquire(lk);
553 }
```

Awakened by another process (transitions to RUNNABLE),
and then selected by the scheduler()

- sleep()
 - Release a lock and sleep on the channel.
 - Reacquires lock when awakened.

wakeup()

```
$ vim kernel/proc.c
```

```
573 void
574 wakeup(void *chan)
575 {
576     struct proc *p;
577
578     for(p = proc; p < &proc[NPROC]; p++) {
579         if(p != myproc()){
580             acquire(&p->lock);
581             if(p->state == SLEEPING && p->chan == chan) {
582                 p->state = RUNNABLE;
583             }
584             release(&p->lock);
585         }
586     }
587 }
```

Wake up the SLEEPING process by just switching the process's state to RUNNABLE.
scheduler() now able to switch-in this process

- wakeup()
 - Wake up all processes sleeping on the channel.

sleep() & wakeup()

Applications in xv6

sleeplock

- Allow processes to yield the CPU while waiting to acquire the lock, enabling efficient resource utilization during lengthy operations such as disk I/O.

```
$ vim kernel/sleeplock.c
```

```
21 void
22 acquiresleep(struct sleeplock *lk)
23 {
24     acquire(&lk->lk);
25     while (lk->locked) {
26         sleep(lk, &lk->lk);
27     }
28     lk->locked = 1;
29     lk->pid = myproc()->pid;
30     release(&lk->lk);
31 }
```

```
$ vim kernel/sleeplock.c
```

```
33 void
34 releasesleep(struct sleeplock *lk)
35 {
36     acquire(&lk->lk);
37     lk->locked = 0;
38     lk->pid = 0;
39     wakeup(lk);
40     release(&lk->lk);
41 }
```

sleeplock vs spinlock

Feature	Spinlocks	Sleeplocks
CPU usage while waiting	Does not yield CPU	Yields CPU
Interrupt handling	Disabled during ownership	Remain enabled
Optimal use case	Short critical sections	Lengthy operations (e.g., disk I/O)
Resource efficiency	Wastes CPU cycles during contention	More efficient CPU usage during contention

pause

```
$ vim kernel/sysproc.c
```

```
67  uint64
68  sys_pause(void)
69  {
70      int n;
71      uint ticks0;
72
73      argint(0, &n);
74      if(n < 0)
75          n = 0;
76      acquire(&tickslock);
77      ticks0 = ticks;
78      while(ticks - ticks0 < n){
79          if(killed(myproc())){
80              release(&tickslock);
81              return -1;
82          }
83          sleep(&ticks, &tickslock);
84      }
85      release(&tickslock);
86      return 0;
87  }
```

- `pause()` system call make a process to sleep for a period of time.

pause

- `pause()` system call make a process to sleep for a period of time.

```
$ vim kernel/sysproc.c
```

```
67  uint64
68  sys_pause(void)
69  {
70      int n;
71      uint ticks0;
72
73      argint(0, &n);
74      if(n < 0)
75          n = 0;
76      acquire(&tickslock);
77      ticks0 = ticks;
78      while(ticks - ticks0 < n){
79          if(killed(myproc())){
80              release(&tickslock);
81              return -1;
82          }
83          sleep(&ticks, &tickslock);
84      }
85      release(&tickslock);
86      return 0;
87  }
```

```
$ vim kernel/trap.c
```

```
164  void
165  clockintr()
166  {
167      if(cpuid() == 0){
168          acquire(&tickslock);
169          ticks++;
170          wakeup(&ticks);
171          release(&tickslock);
172      }
173
174      ...
177      w_stimecmp(r_time() + 100...
178  }
```

pipe

```
$ vim kernel/sysfile.c
```

```
477  uint64
478  sys_pipe(void)
479  {
480      uint64 fdarray; // user pointer to array of ..
481      struct file *rf, *wf;
482      int fd0, fd1;
483      struct proc *p = myproc();
484      ...
485      argaddr(0, &fdarray);
486      if(pipealloc(&rf, &wf) < 0)
487          return -1;
488      fd0 = -1;
489      if((fd0 = fdalloc(rf)) < 0 || (fd1 = ...
490      ...
496      if(copyout(p->pagetable, fdarray, (char*)&fd0,
497          copyout(p->pagetable, fdarray+sizeof(fd0),
498      ...
504      return 0;
505  }
```

- pipe is a communication channel used for data transfer between processes.
 - One process can write data to the pipe, while another process can read it.
- The pipe operates in a half-duplex manner.
 - Data flows in one direction(writer -> reader) only.

pipe

```
$ vim kernel/pipe.c
```

```
76  int
77  pipewrite(struct pipe *pi, uint64 addr, int n)
78  {
...
82      acquire(&pi->lock);
83      while(i < n){
...
88          if(pi->nwrite == pi->nread + PIPESIZE){ ...
89              wakeup(&pi->nread);
90              sleep(&pi->nwrite, &pi->lock);
91          } else {
93              if(copyin(pr->pagetable, &ch, addr + i, ...
94                  break;
95              pi->data[pi->write++ % PIPESIZE] = ch;
96              i++;
97          }
98      }
99      wakeup(&pi->nread);
100     release(&pi->lock);
```

- `pipewrite()` is the function that writes data from the user space into the pipe buffer.
 - Uses producer-consumer model
- If the pipe buffer is full, the writer process sleeps on `nwrite`.
 - And the reader is woken up.
 - If not, the writer starts writing data into the pipe buffer.
- Finally, wake up the reader after writing to the buffer.

pipe

```
$ vim kernel/pipe.c
```

```
105 int
106 piperead(struct pipe *pi, uint64 addr, int n)
107 {
112     acquire(&pi->lock);
113     while(pi->nread == pi->nwrite && ...){
114         ...
118         sleep(&pi->nread, &pi->lock); //DOC: ...
119     }
120     for(i = 0; i < n; i++); //DOC: piperea ...
121     if(pi->nread == pi->nwrite)
122         break;
123     ch = pi->data[pi->nread++ % PIPESIZE];
124     if(copyout(pr->pagetable, addr + i, ...
125         break;
126 }
127 wakeup(&pi->nwrite);
128 release(&pi->lock);
128 return i;
129 }
```

- `piperead( )` is the function that reads up to `n` bytes from the pipe buffer into user space.
- If the pipe buffer is empty, and the writer is still open, the reader process sleeps on `nread`.
 - If not, the reader starts reading up data from the pipe buffer.
- Finally, wake up the writer after reading from the buffer.

Thank you

